Game System: OOP Assignment 5

Solution Explanation Document

This document explains, how I designed and implemented the BattleZone game backend system. It covers my use of abstraction and interfaces, exception handling, and Java Collections, with references to the actual code I wrote.

Part 1 – Abstraction and Interfaces

1.1 The Abstract Class: GameCharacter

The foundation of the entire system is an abstract class called GameCharacter. An abstract class in Java is a class that cannot be instantiated on its own – it exists purely as a blueprint that other classes must extend. I chose this design because every character in the game shares a set of common properties and behaviours, but the exact way each behaviour works differs depending on the character type.

GameCharacter holds three protected fields that every character needs:

- name – a String that identifies the character.
- health – an int representing the character's current hit points.
- energy – an int representing the character's available resource for performing actions.

The class declares three abstract methods that each subclass must implement in its own way:

```
public abstract void attack(GameCharacter target)
    throws InsufficientEnergyException, InvalidActionException;

public abstract void defend();

public abstract void useAbility(GameCharacter target)
    throws InsufficientEnergyException, InvalidActionException;
```

By declaring these methods as abstract, I enforced a contract: any concrete class that extends GameCharacter must provide its own implementation. This is the core idea of abstraction – hiding the implementation details behind a shared interface that the rest of the system can rely on.

The class also provides two concrete methods that are the same for all characters:

- takeDamage(int damage) – reduces health by the given amount, ensuring it never goes below zero.
- isDeath() – returns true if health has reached zero.

Additionally, I overrode the standard Java methods hashCode(), equals(), and toString() inside GameCharacter. The equals() and hashCode() methods compare characters based on name, health, and energy, which ensures consistent behaviour when characters are stored in collections. The toString() method produces a readable summary such as:

Thor - [EN: 95, HP: 115]

1.2 Concrete Character Classes

Warrior

The Warrior class extends GameCharacter and represents a melee fighter. Its stats are initialised in the constructor with 150 HP and 100 energy. The three abstract methods are implemented as follows:

- attack() – deals 20 damage to the target and costs 5 energy.
- defend() – sets an internal boolean flag (defending = true), indicating that the Warrior has taken a defensive stance. The next incoming attack will deal 50% less damage.
- useAbility() – executes 'Shield Slam', which deals 30 damage and costs 10 energy.

The defend logic is handled inside the attacker's attack() method by checking whether the target is defending. If so, the damage is halved and the defending flag is reset. I used the isDefending() and changeDefending() helper methods to read and toggle this flag. This decision keeps the damage calculation in one place (the attacker's method), making it easier to trace the flow of a combat round.

Mage

The Mage class also extends GameCharacter and represents a spell-casting character. Its stats are 100 HP and 200 energy. The Mage has higher energy reserves than the Warrior

because magic abilities are more energy-intensive. The three abstract methods are implemented as follows:

- attack() – deals 15 damage and costs 8 energy. This is slightly weaker but also slightly more costly than the Warrior's attack, reflecting the trade-off between physical and magical combat.
- defend() – raises a magic barrier. Instead of halving damage, the barrier absorbs a flat 20 points of incoming damage. I implemented this using Math.max(0, attDmg - 20) to ensure damage never becomes negative.
- useAbility() – executes 'Fireball', dealing 35 damage at a cost of 15 energy. This is the highest single-target damage output in the base system.

One important correction I made to the original code was fixing the Mage's stats, which had been incorrectly copied from the Warrior (20 damage / 5 energy and 30 damage / 10 energy). I updated them to the values specified in the assignment.

1.3 Interfaces for Special Abilities

Three interfaces define optional abilities that characters can have on top of their basic actions:

Interface	Method Signature
Healable	void heal()
MagicUser	void castSpell(String spellName)
Upgradeable	void upgrade(String stat) throws InvalidActionException

Interfaces in Java allow a class to promise that it provides certain behaviour, regardless of which class it inherits from. This is especially useful when a class needs to combine abilities from multiple sources, since Java does not allow a class to extend more than one class but does allow it to implement multiple interfaces.

I also updated the Upgradeable interface to declare throws InvalidActionException in its method signature. This was necessary because Paladin's upgrade() implementation needs to throw this checked exception when an invalid stat name is provided. Without the declaration in the interface, Java would have rejected the implementation as a contract violation.

1.4 Multiple Inheritance via Interfaces: Paladin

To satisfy the requirement of a third character class that uses multiple interfaces, I created the Paladin class. It extends GameCharacter and implements both Healable and Upgradeable:

public class Paladin extends GameCharacter implements Healable, Upgradeable {

Its constructor initialises 130 HP and 150 energy – a balanced character positioned between the tanky Warrior and the fragile but powerful Mage. The abstract methods are implemented with the following behaviour:

- attack() – deals 15 damage plus any accumulated attack bonus, costs 5 energy.
- defend() – raises a holy shield, halving the next incoming attack.
- useAbility() – executes 'Holy Strike', dealing 25 damage at a cost of 12 energy.

The two interface methods have the following implementations:

heal() – from Healable

The heal() method restores 30 HP and costs 10 energy. Unlike attack and useAbility, heal() does not throw a checked exception. Instead, it prints a message and returns early if the Paladin lacks sufficient energy. I chose this design because healing is a self-targeted action that does not cause the same category of game-breaking errors as attacking a dead target.

```
public void heal() {
   int healAmount = 30;
   int cost = 10;
   if (this.energy < cost) {
      System.out.println(this.name + " tried to heal but has insufficient energy!");
      return;
   }
   this.health += healAmount;
   this.energy -= cost;
}
```

upgrade(String stat) – from Upgradeable

The upgrade() method accepts a stat name as a String and applies a bonus depending on which stat is chosen. Valid values are 'strength', 'health', and 'energy'. Any other value causes the method to throw an InvalidActionException, which is how I satisfy the assignment's requirement to trigger that exception during an upgrade attempt with an invalid stat.

```
public void upgrade(String stat) throws InvalidActionException {
  switch (stat.toLowerCase()) {
    case "strength": this.attackBonus += 5; break;
    case "health":   this.health += 20;    break;
    case "energy":   this.energy += 20;    break;
    default:
      throw new InvalidActionException(
        "Invalid stat \"" + stat + "\"! Valid: strength, health, energy.");
  }
}
```

Part 2 – Exception Handling

2.1 Custom Checked Exceptions

I created two custom exception classes to represent the specific error conditions that can occur during a battle. Both extend Exception, which makes them checked exceptions – the Java compiler requires any method that can throw them to either declare this in its signature with throws, or handle it with a try-catch block.

Exception Class	When It Is Thrown
InsufficientEnergyException	A character tries to attack or use an ability, but does not have enough energy to pay the cost.

InvalidActionException	A character attempts an illegal action, such as attacking a target whose health is already zero, or upgrading a stat that does not exist.

Both classes have a simple structure: a single constructor that accepts a String message and passes it to the parent Exception constructor via super(message). This allows the caller to include a meaningful description of what went wrong, which is then available via getMessage() when the exception is caught.

```
public class InsufficientEnergyException extends Exception {
   public InsufficientEnergyException(String message) {
      super(message);
   }
}
```

2.2 Methods That Throw Exceptions

Every attack() and useAbility() method in Warrior, Mage, and Paladin follows the same validation pattern before performing any action:

1. First, check if the character has enough energy. If not, throw InsufficientEnergyException with a message showing the required and available amounts.
2. Second, check if the target is already dead. If so, throw InvalidActionException with a message identifying the target.
3. Third, if both checks pass, apply the defend mitigation if relevant, deal the damage, and deduct the energy cost.

This ordering matters: it is more informative to tell a player they have no energy before telling them the target is dead, since the energy issue is a resource management problem that the player can address in the next turn. The dead-target check is more of a logic guard against programming errors in the simulation.

2.3 Exception Handling in the Simulation (Arena.java)

The Arena class demonstrates all the exception handling patterns required by the assignment. Every action in the simulation is wrapped in its own try-with-resources block

that writes to game.log. This structure means that even if one action throws an exception, the catch block handles it gracefully and the game continues to the next action.

Try-with-resources

The try-with-resources statement automatically closes the FileWriter after each block, whether the block completes normally or throws an exception. This prevents resource leaks even in error cases, which is a key requirement of robust software design:

```
try (FileWriter log = new FileWriter("game.log", true)) {
   String action = thor.getName() + " attacks " + loki.getName();
   thor.attack(loki);
   history.add(action);
   log.write(action + "\n");
} catch (InvalidActionException | InsufficientEnergyException e) {
   System.out.println("Action error: " + e.getMessage());
} catch (IOException e) {
   System.out.println("Log error: " + e.getMessage());
} finally {
   System.out.println("Game state saved – character health/energy logged.");
}
```

Multi-catch Block

The first catch block uses the multi-catch syntax (separated by |) to handle both custom game exceptions in a single handler. This is appropriate here because both exceptions indicate that a game action failed, and the response is the same in both cases – log the error message and continue. There is no need to write two separate catch blocks with identical code.

Separate IOException Catch

The second catch block handles IOException separately. This is a different category of error – it means the log file could not be written, which is an infrastructure problem rather than a game logic problem. Keeping it separate makes the error reporting clearer.

The finally Block

The finally block runs after every action attempt, regardless of whether it succeeded or failed. I used it to print the 'Game state saved' message as required by the assignment. This block is guaranteed to execute, making it the correct place for cleanup and logging that must always occur.

The Three Triggered Error Scenarios

The simulation deliberately triggers all three required error scenarios to demonstrate that the exception handling works correctly:

- **Error 1 – Attack on a dead target:** Loki's health is set to zero with takeDamage(9999), and then Thor attempts to attack him. The isDeath() check inside attack() detects this and throws InvalidActionException.
- **Error 2 – Insufficient energy for ability:** Kratos' energy is forced to 3 (below the Shield Slam cost of 10) by directly accessing the protected field from within the same package. The energy check inside useAbility() then throws InsufficientEnergyException.
- **Error 3 – Invalid upgrade stat:** Odin calls upgrade("luck"), which is not a valid stat. The default branch of the switch statement throws InvalidActionException.

In all three cases, the error is caught and printed, the failed action is recorded in the history list with a [FAILED] prefix, and the simulation continues without crashing.

Part 3 – Collections Framework

The Arena class manages game data using three different Java collections. Each was chosen for a specific reason based on the requirements of the data it holds.

3.1 List – Action History

```
// List preserves the chronological order of every action taken in the match.

List<String> history = new ArrayList<>();
```

I used an ArrayList to store a String record of every action taken during the simulation. The key requirement here is order – the final report needs to display the last three actions in the sequence they occurred. An ArrayList guarantees that elements stay in insertion order and

supports index-based access, making it straightforward to retrieve the last N items with a simple loop over the final indices.

Failed actions are also recorded with a [FAILED] prefix, so the history gives a complete picture of what happened, including the errors that were handled. This would be essential in a real game for features like a match replay or an audit log.

3.2 Set – Unique Players

```
// Set prevents the same player from joining a match twice.
Set<Player> playerSet = new HashSet<>();
```

I used a HashSet to store the players in a match. The defining property of a Set is that it does not allow duplicates. When I attempted to add a second Player object with the same ID as an existing player (the impostor), the HashSet silently rejected it and returned false from the add() call. No exception is thrown – the duplicate is simply not inserted.

This works because of the equals() and hashCode() implementations in the Player class. Both methods are based exclusively on the playerId field:

```
@Override
public boolean equals(Object obj) {
   if (this == obj) return true;
   if (obj == null || getClass() != obj.getClass()) return false;
   Player other = (Player) obj;
   return this.playerId == other.playerId;
}

@Override
public int hashCode() {
   return Objects.hash(playerId);
}
```

When HashSet checks whether a new element already exists, it first compares hash codes. If two objects have the same hash code, it then calls equals(). By basing both methods on playerId, I ensure that two Player objects are considered identical if and only if they share the same ID, regardless of their nickname or character.

3.3 Map<Integer, GameCharacter> – Player ID to Character

```
// Map gives O(1) character lookup by player ID.

Map<Integer, GameCharacter> playerMap = new HashMap<>();
```

I used a HashMap to map each player's integer ID to their GameCharacter object. A HashMap provides constant-time (O(1)) lookup: given a key, it can return the associated value almost instantly, regardless of how many entries the map contains. This makes it the ideal structure for fast, random-access lookups by ID.

In the final report, I iterate over the map's entry set to print the status of each character:

```
for (Map.Entry<Integer, GameCharacter> entry : playerMap.entrySet()) {

   GameCharacter ch = entry.getValue();

   System.out.println("ID " + entry.getKey() + " -> " + ch);

}
```

This approach is clean and readable: getKey() gives the player ID and getValue() gives the character, whose toString() method (inherited from GameCharacter) formats the health and energy neatly.

3.4 Why These Three Collections?

Collection	Reason for Choice
ArrayList<String>	Maintains insertion order; index access allows easy retrieval of the last N actions; best general-purpose List for sequential data.
HashSet<Player>	Automatically rejects duplicates based on equals()/hashCode(); O(1) average add and contains operations.

HashMap<Integer, GameCharacter>	O(1) average get() by key; ideal for ID-to-object mapping where fast lookup by a unique identifier is required.

Part 4 – Game Simulation (Arena)

The Arena class brings together all four parts of the assignment in a single main() method. The simulation follows a clear sequence:

1. Four characters are created: Thor (Warrior), Loki (Mage), Odin (Paladin), and Kratos (Warrior).
2. Four Player objects wrap these characters with IDs and nicknames.
3. The players are added to the HashSet and the HashMap. A duplicate of Player 1 is also added to demonstrate deduplication.
4. A series of actions is performed, each within its own try-with-resources block. Successful actions are logged to game.log and recorded in the history list. Failed actions are caught, logged to the console, and recorded with a [FAILED] prefix.
5. Three error scenarios are triggered deliberately: attacking a dead target, using an ability without enough energy, and upgrading with an invalid stat.
6. The final report prints the action history (size and last 3 entries), all unique players, and the health and energy of every character.

The key design principle of the simulation is that errors must not crash the game. By wrapping each action independently, a failure in one round has no effect on any subsequent round. This mirrors how a real game server would need to handle unexpected player actions.

Conclusion

This project demonstrates how the three core concepts – abstraction, exception handling, and collections – work together to create a robust and maintainable system.

Abstraction through GameCharacter and the interfaces ensures that new character types can be added without changing the existing combat logic. Exception handling ensures that invalid actions are caught and reported without interrupting the game. The collections ensure that player data is managed efficiently, with no duplicates and fast access patterns.

If I were to extend this project further, I would consider separating the defend logic out of the attacker's methods and instead overriding takeDamage() in each subclass. This would make adding new character types even easier, as no existing class would need to be modified to handle a new character's defence mechanic. I would also explore adding a persistent database layer using a Map<Integer, List<String>> to store the full match history per player ID.